

FreeRTOS 기반 차량 다중 ECU 네트워크 및 CAN-LIN Gateway 개발

차량 제어용 CAN 네트워크와 바디 전장용 LIN 네트워크를 연결하고,
Gateway ECU 가 실시간 라우팅·상태 감시·오류 처리를 수행하는 임베디드 시스템

FreeRTOS

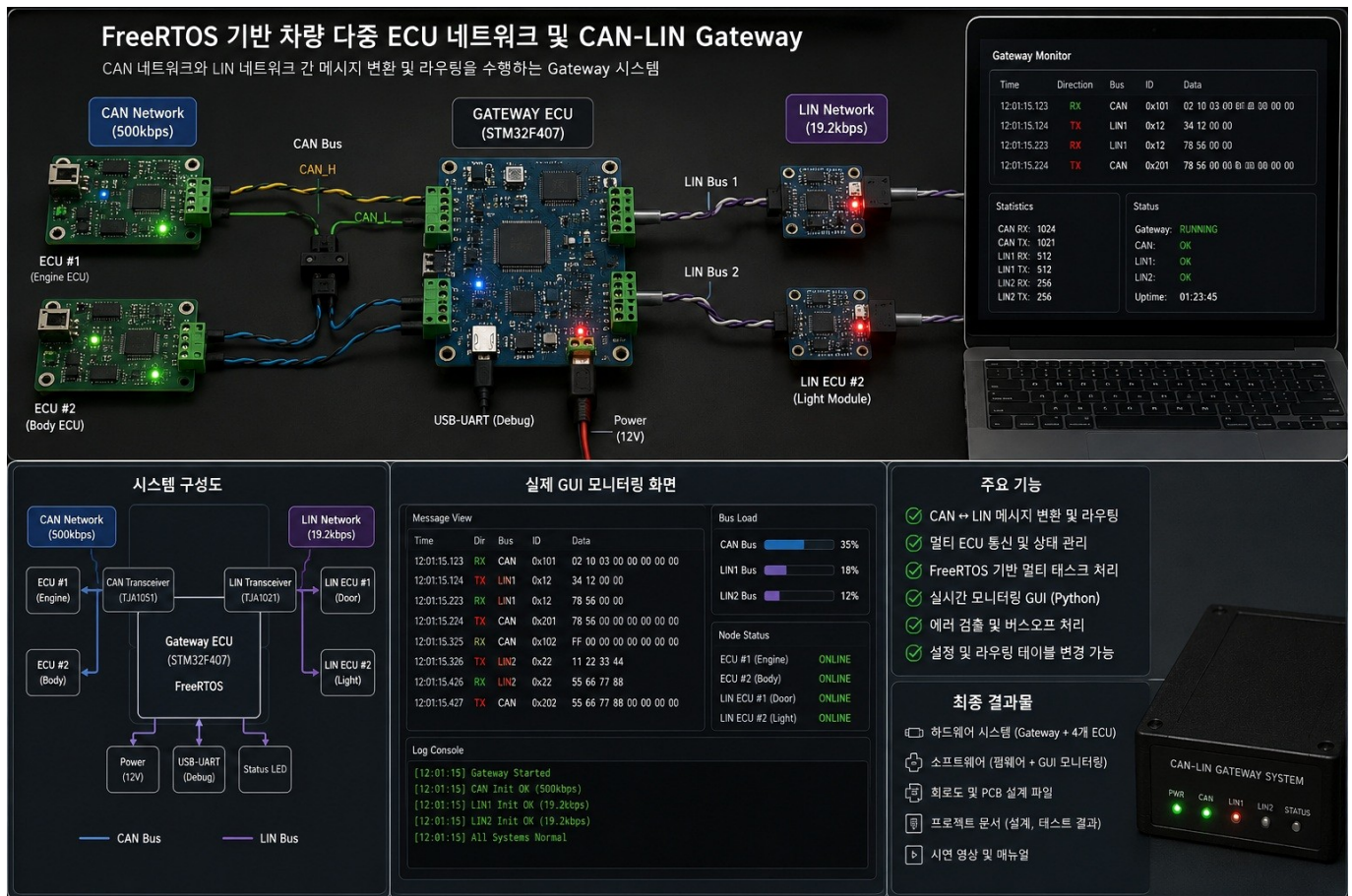
Task · Queue · ISR

Vehicle Network

CAN 500k / LIN 19.2k

3~4 인 / 8~10 주

MVP 부터 단계 확장



※ 위 이미지는 최종 결과를 방향을 설명하기 위한 콘셉트 이미지입니다. 실제 개발보드는 NUCLEO-F446RE 기준으로 구성합니다.

01 왜 이 프로젝트인가?

목표는 “여러 기술을 엮은 작품”이 아니라, 실제 차량 네트워크의 역할 분리와 통합 과정을 경험하는 것입니다.

한 줄 목표 | CAN 기반 주요 ECU 와 LIN 기반 바디 ECU 를 Gateway ECU 가 연결하고, FreeRTOS 로 실시간 통신·라우팅·진단을 안정적으로 처리한다.

프로젝트에서 얻고 싶은 경험

- STM32 에서 CAN·UART/LIN·Timer·GPIO 등 주변장치를 직접 다루는 펌웨어 개발 경험
- FreeRTOS 의 Task, Queue, Mutex, Task Notification 등을 실제 필요에 따라 적용하는 경험
- 여러 ECU 가 각자 역할을 맡고 통신 규격에 맞춰 동작하는 분산 임베디드 시스템 통합 경험
- Timeout, Alive Counter, Checksum, Node Offline 등 정상 동작 외의 오류 상황까지 설계·검증하는 경험
- DBC, LIN Schedule, Routing Table, Test Case 등 “코드 밖의 설계 산출물”까지 남기는 경험

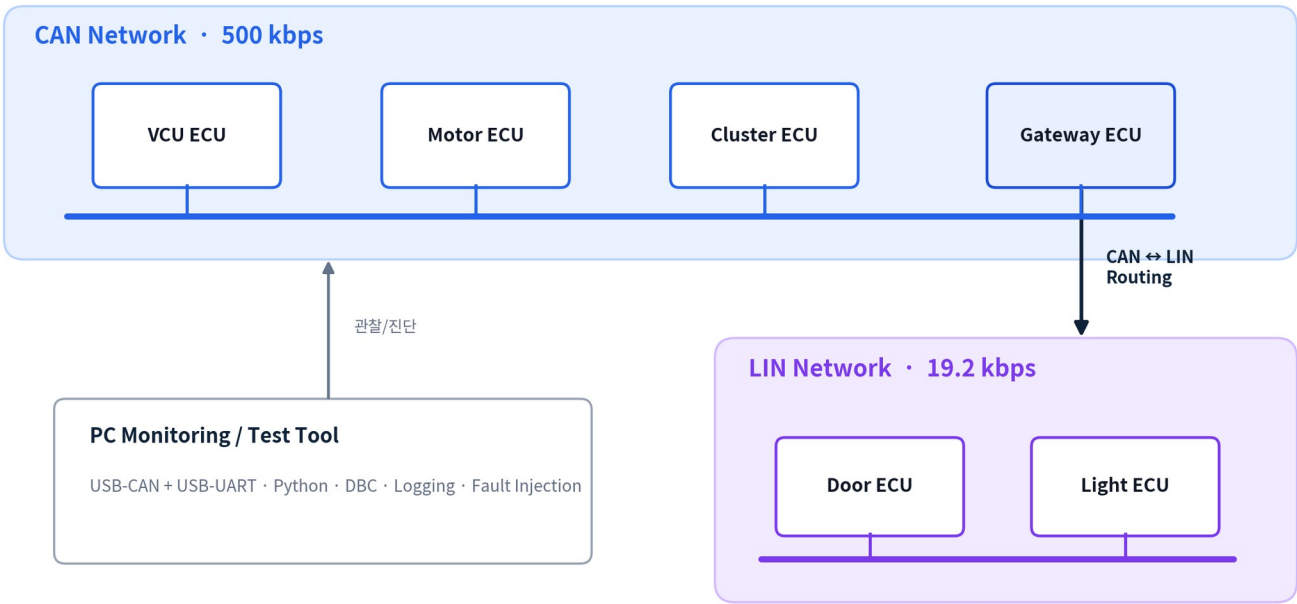
왜 포트폴리오에 유리한가

흔한 프로젝트	이번 프로젝트에서 보여줄 것
단순 CAN 실습	CAN ↔ LIN 간 메시지 변환·주기 차이·라우팅 정책까지 설명 가능
단일 MCU 작품	VCU / Motor / Cluster / Gateway / Door / Light ECU 의 역할과 인터페이스를 구분
RTOS 사용 경험	ISR → Queue → Task 구조, 우선순위와 Queue Overload 를 실제 문제와 연결
정상 동작 데모	통신 단절·체크섬 오류·노드 탈락 등 Fault 시나리오와 Fail-safe 까지 검증

성공 기준: “기능 개수”보다 CAN-LIN Gateway 의 핵심 경로가 안정적으로 동작하고, 왜 그렇게 설계했는지 팀원 모두가 설명할 수 있는 상태.

02 전체 시스템 구조

CAN 은 주요 차량 상태·제어, LIN 은 Door/Light 같은 저속 바디 전장, Gateway 는 두 네트워크의 연결점입니다.



기본 ECU 역할

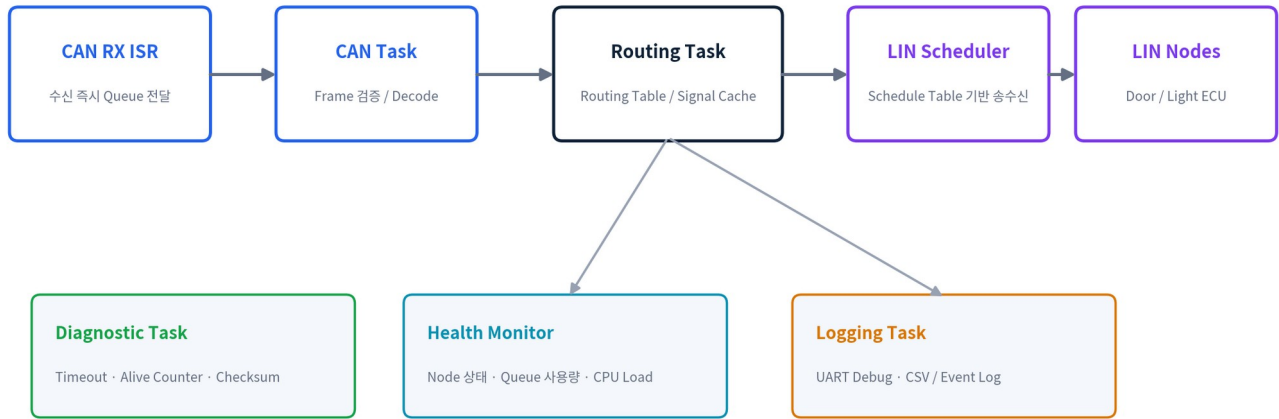
ECU	핵심 역할
VCU ECU	Gear / Brake / Accelerator 입력, Vehicle State, 주요 CAN 상태 송신
Gateway ECU	CAN↔LIN Routing, Signal Cache, Rate Conversion, 통신 상태 감시, 진단
Door ECU	LIN Slave, Door Open/Close·Lock 상태 및 명령 처리
Light ECU	LIN Slave, Head/Reverse/Turn/Hazard Lamp 상태 및 출력
Motor ECU · 확장	Torque Request 수신, PWM/Encoder, RPM·Fault 송신
Cluster ECU · 확장	Speed / RPM / Gear / Door / Warning 표시

MVP 는 VCU → CAN → Gateway → LIN → Door/Light 의 4~5 노드 구성으로 시작하고, 안정화 후 Motor/Cluster ECU 를 붙입니다.

03 Gateway ECU + FreeRTOS 설계

FreeRTOS 는 “썼다”가 아니라, 서로 다른 통신과 주기 작업을 안전하게 분리하기 위해 사용합니다.

Gateway ECU 내부 처리 흐름



원칙: ISR은 짧게, 실제 처리는 Task Context에서 수행 / Queue · Mutex · Task Notification은 필요한 곳에만 사용

Gateway 가 실제로 해결해야 하는 문제

문제	설계 방향
주기 차이	예: CAN Speed 10ms → 최신값 Cache → LIN 100ms Schedule 에 맞춰 전달
동시 처리	CAN 수신, LIN Scheduler, Routing, 진단, Logging 이 동시에 돌아감
ISR 부담	ISR 에서는 Frame 을 Queue 로 넘기고 빠르게 종료, Decode/Route 는 Task 에서 처리
오류 감지	CAN/LIN Timeout, Alive Counter, Checksum, Node Offline, Queue Overflow 감지
확장성	if 문 나열 대신 Routing Table / LIN Schedule 을 설정값으로 분리

면접에서 설명할 핵심: “10ms CAN 메시지를 왜 100ms LIN 으로 그대로 전달하지 않았는가?”, “ISR 에서 왜 Routing 까지 하지 않았는가?”, “노드가 멈췄을 때 어떻게 검출했는가?”

04 주요 기능과 시연 시나리오

완성 후에는 버튼 몇 번과 케이블 한 번만 뽑아도 프로젝트의 핵심이 눈에 보이도록 구성합니다.

핵심 기능

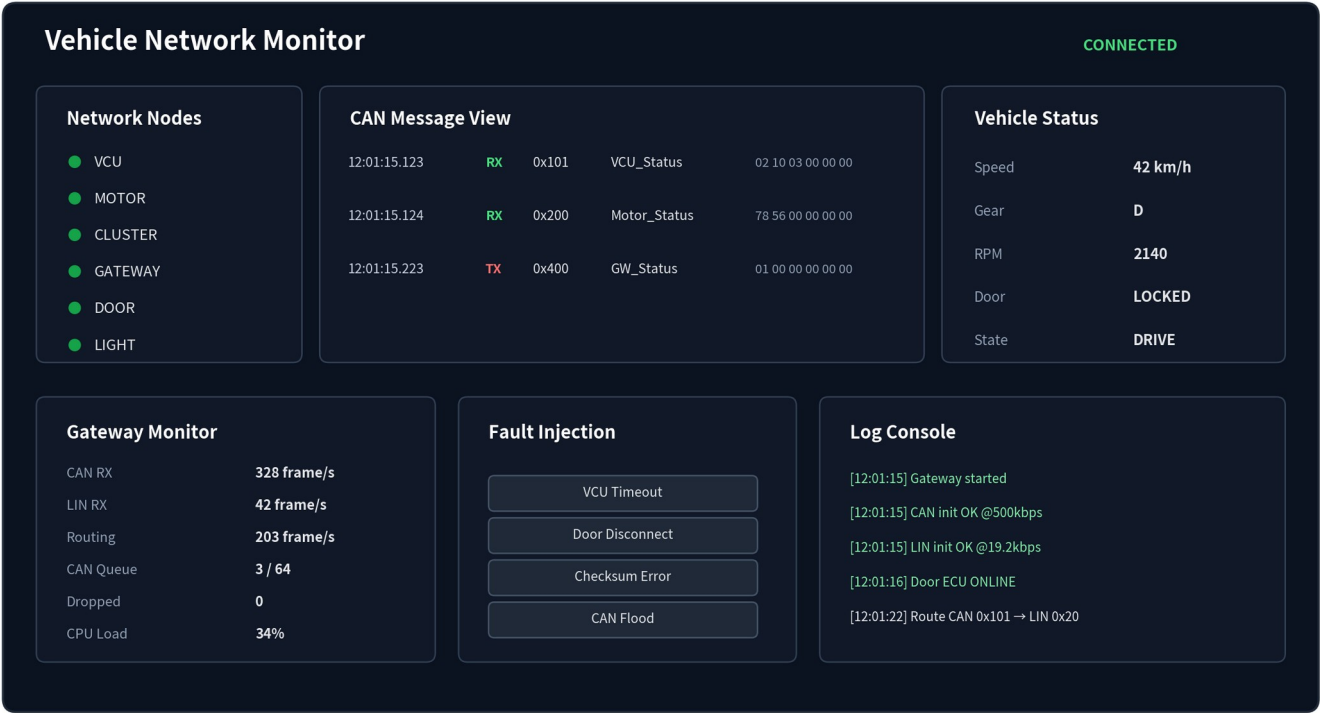
기능	구현 예
CAN → LIN Routing	VCU 의 Gear/Speed/State 를 Gateway 가 받아 Door/Light 명령으로 변환
LIN → CAN Routing	DoorStatus/LightStatus 를 Gateway 가 CAN 메시지로 재구성
LIN Schedule	Gateway 가 LIN Master 로서 Door/Light Frame 을 주기적으로 요청·송신
Network Health	Heartbeat/Timeout/Alive Counter 로 ECU ONLINE/OFFLINE 판단
Fault Injection	PC Tool 에서 Timeout/Checksum/CAN Flood 등을 의도적으로 발생
Logging	CAN/LIN/Gateway 상태를 시간순으로 기록하고 테스트 결과 저장

대표 데모 3 개

01 자동 Door Lock Speed > 20 km/h → CAN 수신 → Gateway 판단 → LIN Door Lock → DoorStatus CAN 복귀	02 Gear R 후진등 Gear = R → VCU CAN → Gateway Route → LIN Light ECU → Reverse LED ON	03 Door ECU 탈락 Door ECU 전원 제거 → LIN Timeout → Gateway Fault → CAN 경고 → GUI OFFLINE
MVP 시연은 모터가 없어도 충분합니다. 통신 경로와 오류 처리부터 완성한 뒤 Motor/Cluster 를 확장합니다.		

05 개발환경과 GUI 모니터링

펌웨어 · PC 도구 · 네트워크 정의 파일을 하나의 Git 저장소에서 함께 관리합니다.



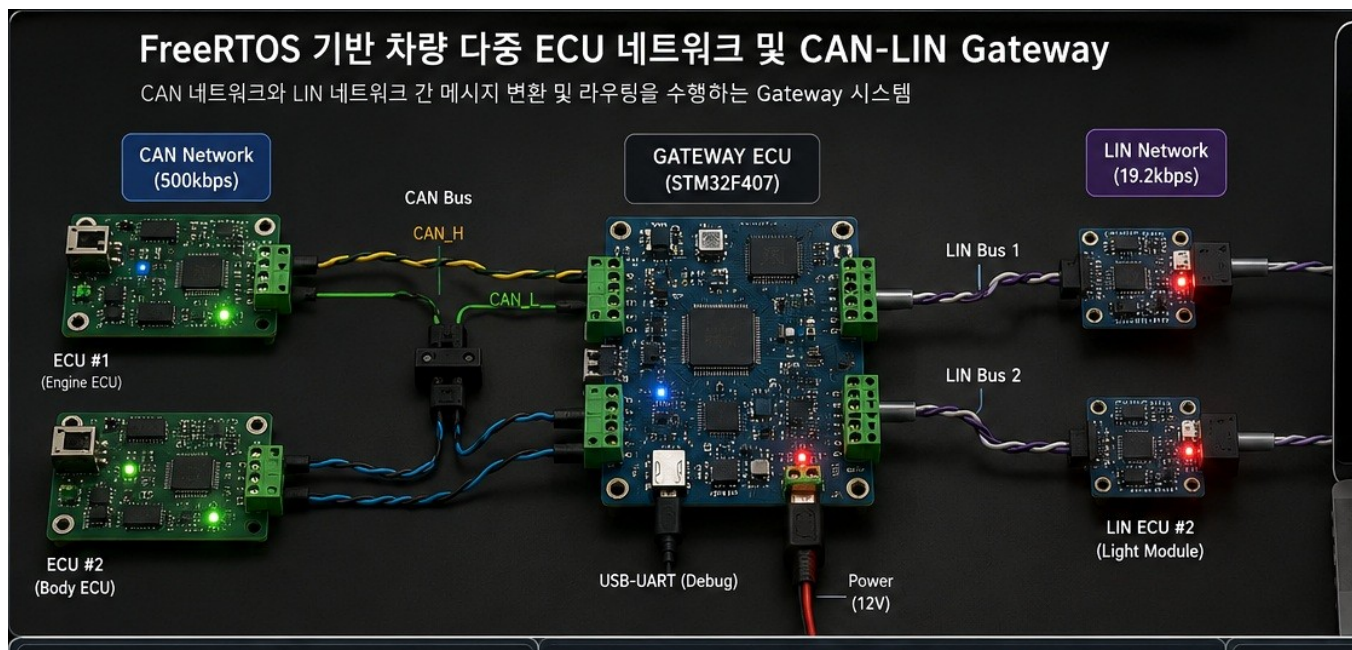
개발 스택

구분	사용 기술
MCU / IDE	STM32F446RE · G071RB / STM32CubeIDE / STM32CubeMX
Firmware	C / STM32 HAL / FreeRTOS
Network	Classical CAN 2.0B 500kbps / LIN 19.2kbps / DBC / LIN Schedule
PC Tool	Python / PySide6 / python-can / cantools / pyserial
Version Control	Git + GitHub / Pull Request / Code Review
Document	Markdown 또는 Notion / draw.io / Test Case · Test Report

GUI에서 보여줄 것: Raw CAN Frame · DBC Signal · ECU Online/Offline · Gateway CAN/LIN RX/TX · Queue 사용량 · Dropped Frame · Fault Injection · Log

06 준비물과 개발 장비

오실로스코프는 구매하지 않습니다. 멀티미터 + CANable + 저가 로직 애널라이저면 MVP 개발에 충분합니다.



※ 대표 구성 이미지입니다. 실제 구매품과 외형은 다를 수 있습니다.

장비	상태	용도
멀티미터 본체	보유	전원 · 저항 · 종단저항 · 단선 확인
멀티미터 테스트 리드/후크	구매	4mm 바나나 타입 우선, 핀/터미널 고정 측정
CANable USB-CAN	구매	CAN Frame 송수신 · 모니터링 · Fault Injection
USB 로직 애널라이저 24MHz 8ch	구매	UART/LIN TX/RX, 디지털 Timing 확인
USB-UART CP2102	구매	Gateway Debug/Status 로그
오실로스코프	비구매	물리계층 파형이 꼭 필요할 때 교육장/학교 장비 활용

CAN 통신 이상 시 기본 점검 순서: 전원 → GND → 전원 OFF 상태 CAN_H↔CAN_L 약 60Ω → MCU CAN_TX/RX → CANable 수신 여부 → 필요 시 오실로스코프.

07 구매 목록과 예상 예산

가격은 2026-09-02 확인값/예상 범위이며 재고·배송·환율에 따라 변동될 수 있습니다.

준비물	수량	예상가	구매/참고 링크
NUCLEO-F446RE	2	약 71,500 원	디바이스마트
NUCLEO-G071RB	2	약 31,800 원	ICBanQ
SN65HVD230 CAN 모듈	2	약 24,200 원	디바이스마트
TJA1021 LIN 모듈	3	약 45,000~75,000 원	LIN 모듈 예시
CANable V2.0	1	약 48,400 원	디바이스마트
24MHz 8ch Logic Analyzer	1	약 13,200 원	디바이스마트
CP2102 USB-UART	1	약 4,600 원	디바이스마트
Breadboard / Jumper / LED / Switch	묶음	약 30,000 원	점퍼선 예시
멀티미터 후크/리드	1 세트	약 3,000~10,000 원	테스트 툴 예시
12V 전원 / DC-DC / 터미널	묶음	약 20,000 원	검색 구매

MVP 예상 예산

MVP 구매비 약 30~33 만 원 VCU + Gateway + Door + Light	여유 포함 권장 약 35 만 원 배송비·예비부품 포함	4 인 기준 약 8~9 만 원/인 MVP 기준
---	--	--

확장형: Motor/Cluster ECU, Encoder Motor, Motor Driver, OLED 등을 추가하면 약 13~15 만 원이 더 들어가며 최종 총예산은 대략 43~48 만 원 수준으로 예상합니다.

08 팀 구성과 10 주 로드맵

각자 메인 담당은 나누되, CAN Matrix·LIN Schedule·Routing Table·통합시험은 팀 공통 산출물로 가져갑니다.

4 인 역할 예시

담당	메인 영역	세부
A	Gateway ECU	FreeRTOS / Routing / CAN-LIN / 진단
B	CAN Network	VCU / CAN Matrix / Motor ECU 확장
C	LIN Network	Door·Light ECU / LIN Slave / Schedule
D	Tool & Test	PySide6 GUI / Logging / Fault Injection / Test
공통	Architecture & Integration	요구사항 / Interface / 코드리뷰 / 통합시험

10 주 로드맵

주차	목표
1 주	요구사항 / Architecture / 역할 / Git 규칙
2 주	STM32 CAN Bring-up / CAN Matrix 초안
3 주	LIN Bring-up / Gateway-LIN Slave 통신
4 주	CAN-LIN Routing MVP 완성
5 주	Door/Light ECU 기능·State 구현
6 주	FreeRTOS Task/Queue 구조 리팩터링
7 주	Timeout / Alive Counter / Checksum / Fault
8 주	GUI Monitor / DBC / Logging
9 주	Fault Injection / Queue 부하 / 통합시험
10 주	문서 / Demo 영상 / 포트폴리오 정리

중요: 8 주차에 처음 합치는 방식은 피하고, 2~4 주차부터 매주 통합 가능한 상태를 유지합니다.

09 최종 결과물과 같이할 사람

완성품뿐 아니라 설계·시험·문제 해결 과정까지 남겨 “우리가 무엇을 고민했고 왜 그렇게 만들었는지” 설명 가능한 상태를 목표로 합니다.

결과물	내용
하드웨어	Gateway + CAN ECU + LIN ECU 네트워크 / 배선 / 상태 LED
펌웨어	VCU · Gateway · Door · Light (+ Motor/Cluster 확장) 소스코드
PC Tool	CAN/LIN 상태 모니터링, DBC Decode, Logging, Fault Injection GUI
설계 문서	System Architecture, CAN Matrix, DBC, LIN Schedule, Routing Table
시험 문서	Requirement ↔ Test Case 연결, Fault Test, 부하 Test, 결과 기록
포트폴리오	GitHub README, Trouble Shooting, 역할/기여도, 2~3 분 Demo 영상

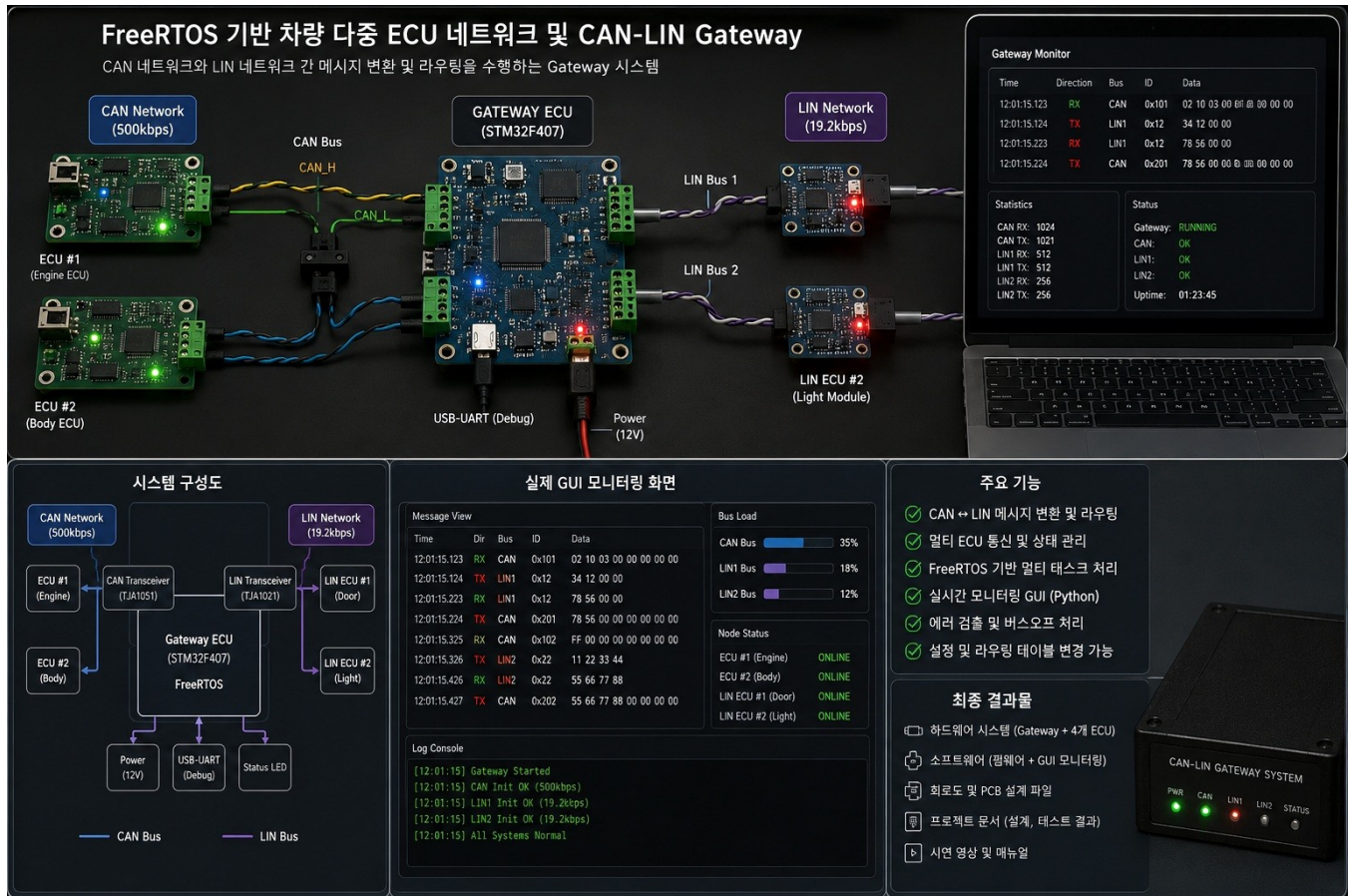
같이하면 좋은 사람

- 임베디드/차량 SW 쪽 포트폴리오를 하나 제대로 남겨보고 싶은 사람
- 모르는 기술이 있어도 각자 맡은 부분을 끝까지 파고, 통합할 때 서로 도와줄 사람
- 매주 짧게라도 진행상황을 공유하고 Git/문서/테스트까지 같이 정리할 사람
- 완벽한 실력보다 “MVP 부터 실제로 움직이게 만든다”는 방식에 동의하는 사람

첫 미팅에서 정할 것 | ① 3~4 인 팀 확정 ② MVP 범위 확정 ③ 역할 1 차 배정 ④ 1 차 구매 목록 확정 ⑤ Git 저장소 / 주 1 회 통합 일정 결정

10 최종 결과물 콘셉트

최종적으로는 하드웨어 네트워크 + Gateway Firmware + PC Monitoring Tool + 설계/시험 문서가 하나의 포트폴리오 패키지로 납품니다.



※ 콘셉트 이미지 내 일부 보드/표기는 시각화용입니다. 실제 BOM 과 구현은 본 제안서의 NUCLEO-F446RE/G071RB 기준을 우선합니다.

프로젝트 완료 시 한 문장: “FreeRTOS 기반 Gateway ECU 를 중심으로 CAN 과 LIN 다중 ECU 네트워크를 구성하고, 실시간 Routing·상태 감시·Fault Handling·GUI 진단까지 구현했다.”

기술 참고: [STM32 NUCLEO-F446RE](#) · [FreeRTOS](#)